

Table.py

Student worksheet - build the shared poker table state

Create a file called table.py. Work through the steps in order. Each part adds one small piece to the final program. The final version should be able to store the community cards and the total pot for a poker hand.

This worksheet does not assume you have implemented cards.py before. You can treat Card as a placeholder object that represents a playing card.

Before you start: Use meaningful names and keep the code readable. Test after each section instead of writing the whole file first.

0. Set up the file

At the top of table.py, add the imports you will need.

```
from dataclasses import dataclass, field
from .cards import Card
```

Hint: field(default_factory=list) creates a fresh empty list for each new Table instance.

1. create the class Table:

Use @dataclass above the class. Add the following attributes:

- community_cards: an empty list by default
- pot: defaults to 0

Checkpoint:

```
table = Table()
print(table.pot)  # expected: 0
```

2. create the method reset(self) (in class Table):

This method should reset the table for a new hand:

- clear all community cards
- reset the pot back to 0

Hint: clear() removes all items from the list without replacing the list object itself.

3. create the method `add_to_pot(self, amount)`:

This method should add chips to the pot:

- raise `ValueError` if amount is negative
- otherwise, increase the pot by the given amount

Checkpoint:

```
table = Table()
table.add_to_pot(50)
print(table.pot) # expected: 50
```

Common mistakes to check

- Using `[]` directly instead of `field(default_factory=list)`, which causes all instances to share the same list.
- Forgetting to reset the pot in the reset method.
- Allowing negative amounts into the pot.
- Replacing the list with a new one instead of clearing it with `clear()`.